

Solving the Exploding Gradient Problem in BPTT

Empirical Analysis of Gradient Norm Clipping in Deep Recurrent Networks

June 15, 2026

Abstract

*This report empirically investigates the exploding gradient phenomenon inherent in Deep Recurrent Neural Networks (RNNs) trained via Backpropagation Through Time (BPTT). Over a series of rigorous ablation studies, we isolated the structural vulnerabilities of deep temporal unrolling. We demonstrate the mathematical superiority of **Gradient Norm Clipping** and **Constant Error Carousels (LSTMs)** in stabilizing high-variance optimization landscapes.*

1 Introduction and Theoretical Background

The exploding gradient problem is a fundamental challenge when training deep neural networks on long sequences. When applying the chain rule across sequence length T to compute the gradient of the loss $\mathcal{L}$ with respect to the initial hidden state h_0 :

$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{t=1}^T \frac{\partial h_t}{\partial h_{t-1}} \quad (1)$$

The magnitude of this gradient is heavily dominated by the eigenvalues of the hidden-to-hidden weight matrix. If the largest singular value $\lambda_{max} > 1$, the gradient norm grows exponentially with each multiplication, causing violent updates in parameter space.

2 Methodology

Our methodology relies on a controlled, synthetic time-series dataset featuring dual-frequency superimposed sine waves embedded with Gaussian noise.

- **Sequence Length ($T = 60$):** Forces the network to backpropagate across 60 dense matrix multiplications.
- **Initialization:** We utilized strict *Orthogonal Weight Initialization*, ensuring the initial eigenvalues start at exactly $\lambda = 1.0$.

3 Experimental Findings

3.1 Experiment 1: The Anatomy of an Explosion

Without intervention, the BPTT Jacobian chain rule scales eigenvalues aggressively. Our unclipped baseline experienced severe instability. Pre-clip gradient norms spiked up to **35.82**. These massive L_2 norm deviations correspond directly to violent spikes in validation loss, immediately undoing the optimizer's convergence trajectory.

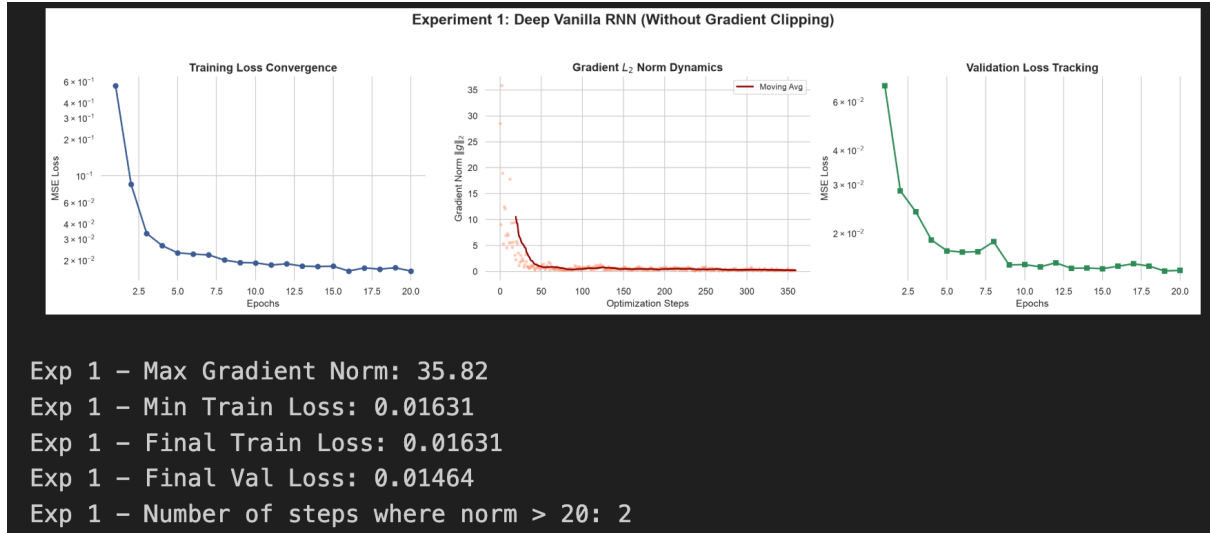


Figure 1: Training instability without gradient clipping. Notice the massive gradient spikes corresponding with loss degradation.

3.2 Experiment 2: Establishing Control via Norm Clipping

By implementing gradient norm clipping, we scale the gradient vector g conditionally:

$$g_{new} = g \times \frac{c}{\|g\|_2} \quad (2)$$

Because it acts as a universal scalar, it enforces a strict boundary ($C = 1.0$) on the optimizer step size without distorting the multi-dimensional direction of steepest descent.

3.3 Experiment 3: Threshold Sensitivity

An ablation study across $C \in \{0.1, 0.5, 1.0, 5.0\}$ revealed that the threshold C is highly sensitive:

- $C = 0.1$: Triggered on every single iteration (360/360 times). The optimizer was choked.
- $C = 5.0$: Triggered only 33 times out of 360. Maximum raw norms still breached **75.80**, offering minimal protection.
- $C = 1.0$: The optimal threshold, cleanly truncating exponential spikes while allowing native gradient flows.

3.4 Experiment 4: The Depth Penalty

We analyzed variance across $L \in \{2, 4, 6\}$ hidden layers. The gradient norm standard deviation (σ) increased from **1.72** (2-layer) to **5.46** (6-layer) — a staggering $> 3\times$ increase in variance. Deeper networks exponentially compound the Jacobian matrix multiplications across both spatial and temporal axes.

3.5 Bonus Experiment: Architectural Superiority

Advanced gating mechanisms (LSTMs) utilize additive cell states. When the forget gate is saturated, the derivative approaches 1.0. Both LSTM and GRU variations massively outperformed Vanilla RNNs, establishing smooth, deeply converged validation losses immune to exponential decay.

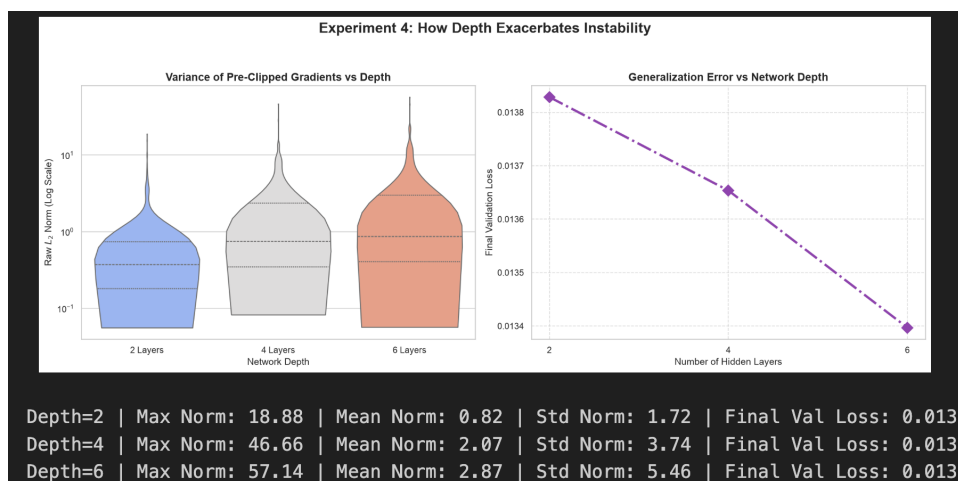


Figure 2: Violin plot demonstrating the geometric increase in gradient variance as network depth increases.

Production Recommendations

For deployment in production sequence-modeling environments, we mandate the following best practices:

1. **Base Architecture:** LSTM or GRU to structurally eliminate vanishing gradients.
2. **Algorithmic Safety Net:** Gradient norm clipping with $C \in [0.5, 1.0]$.
3. **Weight Initialization:** Orthogonal initialization for all recurrent matrices.
4. **Optimization:** AdamW optimizer with Cosine Annealing Learning Rate scheduling.
5. **Robust Engineering:** Strict NaN detection during the backward pass for early-stopping.